1 Java 1.0–1.4 · The foundations · cheat sheet

Java 1.0 – 1.4 · 1996 – 2002 · 16 features

Classes, interfaces, exceptions → Bytecode on a VM with GC → Look inside: reflection, inner classes → A real collections framework → HotSpot, regex, NIO

1 THE OBJECT MODEL

1 Classes and inheritance

One superclass, Object at the root

Single inheritance, Object at the root, virtual by default. final unless designed for extension; compose.

```
final class Circle extends Shape
```

2 Interfaces

A contract with no implementation

Multiple inheritance of type, none of state. Name the interface in signatures; keep each one small.

```
class A implements Runnable, Closeable
```

3 Exceptions

Failure modes in the signature

Error / RuntimeException (unchecked) / Exception (checked, in throws). finally always runs; chain the cause.

```
throw new MyEx("msg", cause);
```

2 THE MACHINE UNDERNEATH

4 JVM and bytecode

Write once, run anywhere (mostly)

javac → class files → any JVM. The class file is the contract; javap -c shows it. JNI (1.1) = native door.

```
javap -c Hello
```

5 Garbage collection

No free(), no dangling pointers

Managed heap; unreachable objects reclaimed. Leaks = forgotten references. No System.gc(); finalize() is gone.

```
java -Xlog:gc -Xmx512m App
```

6 HotSpot JIT

Interpret first, compile what is hot

Default VM since 1.3: interpret, profile, compile hot methods, inline, de-optimise. Warm up, then measure.

```
-XX:+PrintCompilation
```

7 Threads and synchronized

Monitors built into every object

Thread, Runnable, synchronized/wait/notify since 1.0. One lock for all shared state. start(), not run().

```
synchronized void inc() { n++; }
```

3 JAVA 1.1: LOOKING INSIDE

8 Inner and anonymous classes

A class inside a class, capturing it

Static nested, inner (hidden Outer.this), local, anonymous. Default to static; anonymous = old callbacks.

```
new Runnable() { public void run(){} }
```

9 Reflection and JavaBeans

Inspect and call types at run time

Class, Method, Field, Constructor at run time; JavaBeans get/set; dynamic proxies (1.3). Frameworks, not apps.

```
c.getMethod("setX", int.class)
```

10 Serialization

Objects to bytes, with a bill

Serializable + ObjectInput/OutputStream. Gadget-chain attacks: filter (JEP 290) or avoid; prefer JSON.

```
-Djdk.serialFilter=com.acme.*;!*
```

4 JAVA 1.2: COLLECTIONS

11 Collections framework

List, Set, Map: interfaces first

Collection/List/Set/Map interfaces; ArrayList/HashMap/TreeMap implementations. Declare the interface.

```
List<T> xs = new ArrayList<>();
```

12 Iterator, Comparator, utilities

Walk it, order it, wrap it

Iterator.remove() to delete safely; Comparable (natural) vs Comparator (external); Collections.sort etc.

```
Collections.sort(xs, cmp)
```

13 Vector, Hashtable, Enumeration

The 1.0 collections: legacy, avoid

~~DEPRECATED~~

Vector, Hashtable, Stack, Enumeration: locked per call, unsafe for check-then-act. Use ArrayList/HashMap.

```
Vector→ArrayList, Hashtable→HashMap
```

5 JAVA 1.4: THE TOOLBOX

14 Regular expressions

Compile once, match many

Pattern.compile once (static final); Matcher.find/matches/group. String.matches recompiles; (a+)+ backtracks.

```
Pattern.compile(re).matcher(s).find()
```

15 NIO buffers and channels

Buffers, channels, selectors

ByteBuffer (heap/direct) + Channel + Selector; flip() between write and read. NIO.2 (7) adds Files/Path.

```
buf.flip(); ... buf.clear();
```

16 assert and java.util.logging

Check invariants; log with levels

assert cond : msg needs -ea (off by default); java.util.logging levels/handlers. Validate with exceptions.

```
assert x > 0 : "x";  // java -ea
```

Do / don't, the headline features

Classes and inheritance
Do: final unless designed for extension; private fields; equals and hashCode together.
Don't: Don't extend to reuse code (Stack extends Vector exposed insertElementAt). Compose.

Exceptions
Do: Wrap with the original as cause; catch the most specific type you can handle.
Don't: No empty catch, no log-and-rethrow, never return from finally.

JVM and bytecode
Do: Spend ten minutes with javap -c: concat, boxing and synchronized become visible.
Don't: Don't assume behaviour is portable: paths, line endings, charset (UTF-8 default only since 18).

Threads and synchronized
Do: One lock for all shared state, reads included; prefer a private lock object.
Don't: start(), not run(); never lock on String literals or boxed values; Thread.stop throws.

Don't mix these up

Checked vs. unchecked exception
Checked extends **Exception** and must appear in `throws` or be caught. Unchecked extends **RuntimeException** (or Error) and may be ignored by the compiler, not by you.

Interface vs. abstract class
An interface is a **contract**; a class may implement many. An abstract class is a **partial class** with state and constructors; you get exactly one.

== vs. equals()
`==` compares **references** (same object). `equals()` compares **values** as the class defines them. Strings, boxed numbers, collections: always equals().

Inner vs. static nested class
An inner class holds a hidden **Outer.this** and cannot exist without an instance. A static nested class is **just a namespace**. Default to static.

Comparable vs. Comparator
`Comparable` is the type's **own** order (`compareTo` , one per class). `Comparator` is an order supplied **from outside**, as many as you like.

Vector vs. ArrayList
Same idea; `Vector` locks on **every call** and is still unsafe for check-then-act. `ArrayList` is unsynchronised and faster; use j.u.c when sharing.

start() vs. run()
`start()` asks the JVM for a **new thread**, which then calls run(). Calling `run()` yourself runs the body on the **current** thread, synchronously.

final vs. finally vs. finalize()
`final` : no reassignment, no override, no subclass. `finally` : runs on **every exit** from try. `finalize()` : a GC hook, deprecated, never use it.

Java 1.0 fixed the shape of the language: classes, interfaces, exceptions, a virtual machine with garbage collection, threads. The 1.x releases filled in the library: reflection, collections, a real JIT, NIO. Everything later stands on this floor.

Full cards: study guide, chapter 1 · knowledge graph · quiz · practice/